SLE-3: Architectural Design

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM137 Name: Harshal Kamble
Division: B Date: 04 October
SLE-2 Basis: Tic-Tac-Toe BFS vs DFS

---

1. System Title & Short Description

Tic-Tac-Toe Search System (BFS / DFS) — a search-based agent that explores the Tic-Tac-Toe game tree to choose a move for the player. The system takes a 3×3 board state and the player to move as input and explores future positions using either Breadth First Search (BFS) or Depth First Search (DFS). It compares two uninformed search strategies on small, medium and empty-board states, which is a core building block for game-playing agents. This system continues directly from the BFS/DFS implementation and profiling work completed in SLE-2. SLE-3 adds a performance-oriented redesign on top of that baseline: bitboard state encoding, visited-state pruning and symmetry reduction. On the empty board this cuts the search from 549,946 expanded nodes to 765 while selecting exactly the same move.

2. Context Diagram (Level 1)



Figure 1: Context diagram – system boundary and external actors

The **Player / User** supplies the current 3×3 board state and the player to move to the **Tic-Tac-Toe search system**. The system explores the legal future positions internally and returns a **Move output** — the selected move together with its minimax value, the execution time and the node count. No external systems are involved; the whole system runs as a self-contained Python script with no network or API dependency.

3. Container Diagram (Level 2)

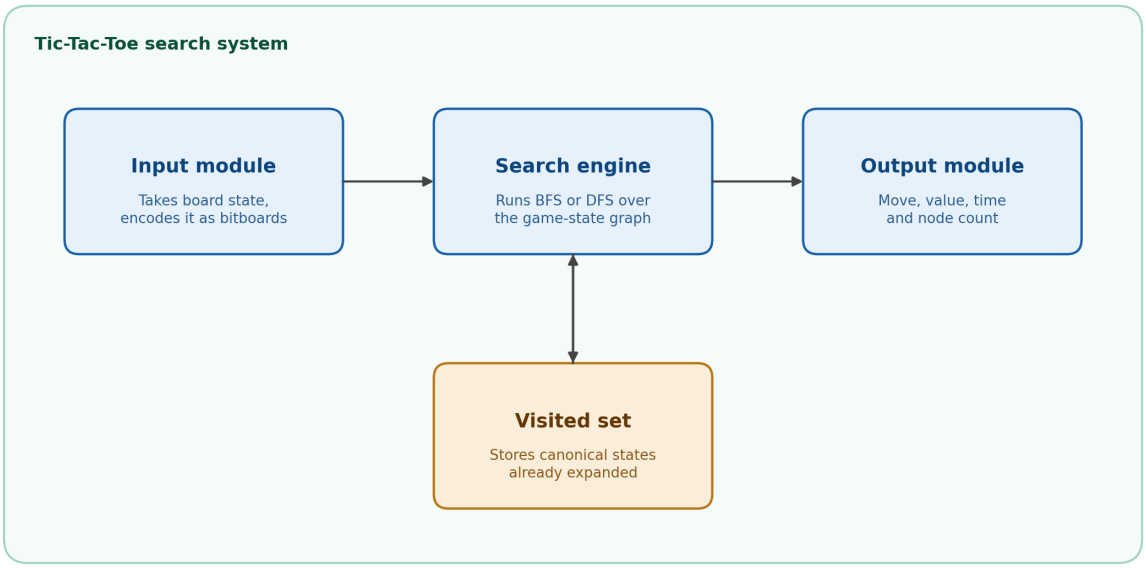


Figure 2: Container diagram – main building blocks inside the system

Explanation of each container:

- **Input module:** Reads the board state and player to move, and encodes the board as two 9-bit bitboards (X and O).
- **Search engine:** Runs the chosen algorithm, BFS or DFS, over the game-state graph and evaluates the positions it finds.
- **Visited set:** Stores the canonical form of every position already expanded, so each position is searched only once (the SLE-3 addition).
- **Output module:** Reports the selected move and its value, along with the time and expanded-node count used for profiling.

4. Component Diagram (Level 3)

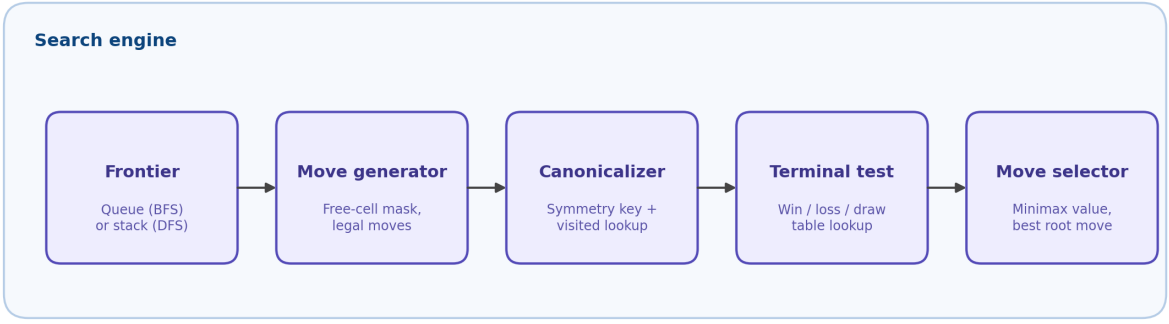


Figure 3: Component diagram – inside the Search engine container

The **Search engine** is the most important container, so it is broken down further. The **Frontier** holds positions waiting to be explored — a queue for BFS (FIFO) or a stack for DFS (LIFO); it is the only structural difference between the two algorithms. The **Move generator** lists the legal moves from the free-cell mask. The **Canonicalizer** turns each successor into its canonical key (the smallest of its eight

rotations/reflections) and checks it against the Visited set, so repeated positions are not expanded again. The **Terminal test** recognises a win, loss or draw with a table lookup. The **Move selector** evaluates the explored positions from the deepest level upwards (X maximises, O minimises) and picks the best root move.

5. Code Level Overview (Level 4)

Main classes / functions:

- **Board state (x, o) :** two 9-bit integers; bit i is set when cell i holds that player's mark. This replaces copying a 9-element list at every node.
- **get_moves() :** free = FULL & ~(x | o); each legal move is peeled off with free & -free, so there is no scan over nine cells.
- **winner() / terminal test :** IS_WIN[mask] is a 512-entry precomputed table over the eight winning-line masks, so a win check is one lookup.
- **canon(x, o) :** returns the smallest key over the 8 board symmetries (4 rotations × reflection) using the precomputed SYM_TABLE; used only when at least 8 cells are empty (SYMMETRY_MIN_EMPTY).
- **bfs() / dfs() :** the same expansion loop; the frontier is a deque popped from the left (BFS) or a list popped from the right (DFS). Each child is looked up in the visited map (seen) and added only once.
- **Backward induction :** positions are evaluated in decreasing depth and the root move is chosen from its children's values.
- **Timing and node counter :** time.perf_counter() records milliseconds (median of repeated runs); the node counter counts positions popped from the frontier, root included.

6. Design Decisions

The same board representation and legal-move logic are shared by BFS and DFS so that the comparison focuses on the search strategy. The frontier is the only structural difference (queue vs. stack), which is why the two algorithms share almost identical surrounding logic (visited-set checks, terminal test, move selection). Timing and node counting are kept outside the core search decision so both algorithms can be compared on the same inputs.

SLE-3 performance decisions:

- **Visited-state pruning:** the same position is reachable through many move orders. The empty board has 549,946 game-tree nodes but only 5,478 distinct positions, so a visited map turns the tree into a graph and removes repeated work. A position's depth is fixed by how many marks are on the board, so BFS and DFS expand the same set of states.
- **Bitboards and lookup tables:** integer operations and two precomputed tables replace list copying and line-by-line scanning, which lowers the cost of every node.
- **Symmetry reduction:** rotated and reflected boards share one canonical key (765 distinct classes from the empty board). Canonicalising costs eight lookups per state, so it is enabled only when at least 8 cells are empty; below that it measured slower (for example about 0.19 ms versus 0.11 ms for BFS with 5 empty cells).

- **Correctness check:** the baseline and every optimized variant were run on each test case with BFS and DFS, and the selected move and minimax value were identical in all cases.
- **Trade-off:** the visited map uses extra memory (at most 5,478 entries here), which is small compared with the repeated work it removes.

7. Profiling Results (SLE-2 baseline and SLE-3 optimized)

7.1 SLE-2 results (as originally recorded)

Test case	BFS time (ms)	DFS time (ms)	BFS nodes	DFS nodes
3 empty cells	0.1094	0.1323	5	4
5 empty cells	1.7291	0.4136	210	209
9 empty cells	2525.3516	1441.2642	549946	549945

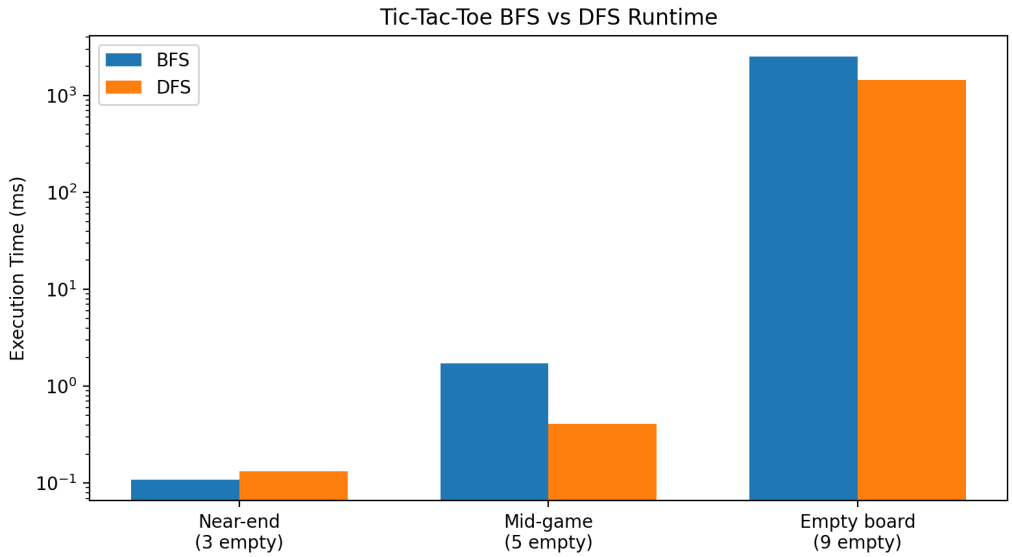


Figure 4: BFS vs DFS observed execution time across the three Tic-Tac-Toe test cases (log scale)

7.2 SLE-3 re-run: baseline vs optimized (same machine, same positions)

Both versions were run back to back in the same environment on the same positions, so the speed-ups are like-for-like. The 5-empty position (X to move) has a game tree of 210 nodes, matching the SLE-2 BFS count; the 3-empty position is a representative position and is not identical to the SLE-2 one. Absolute times differ from Section 7.1 because they were measured on a different machine.

Table 1: Execution time, baseline vs optimized (median of repeated runs)

Test case	BFS baseline (ms)	BFS optimized (ms)	DFS baseline (ms)	DFS optimized (ms)	Speed-up BFS / DFS
3 empty cells	0.0204	0.0218	0.0193	0.0226	0.9× / 0.9×
5 empty cells	0.2818	0.1114	0.2905	0.1111	2.5× / 2.6×
9 empty cells	881.0013	3.2251	890.0381	3.4378	273× / 259×

Table 2: Search nodes expanded

Test case	Baseline nodes	Visited-state pruning	+ Symmetry reduction	Reduction (config used)
3 empty cells	6	6	4	1.0×
5 empty cells	210	82	72	2.6×
9 empty cells	549,946	5,478	765	719×

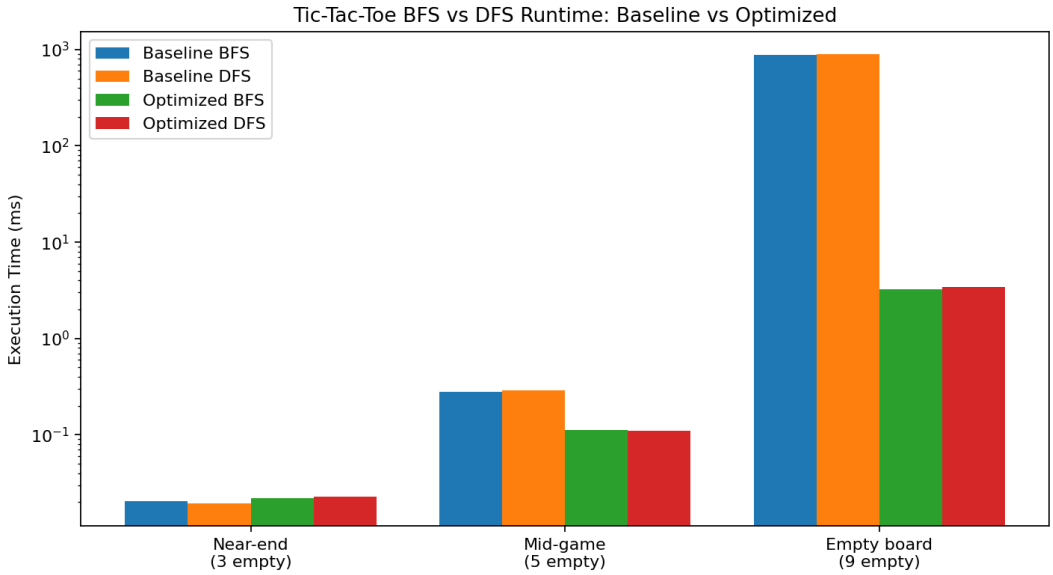


Figure 5: Execution time across the three test cases, baseline vs optimized (log scale)

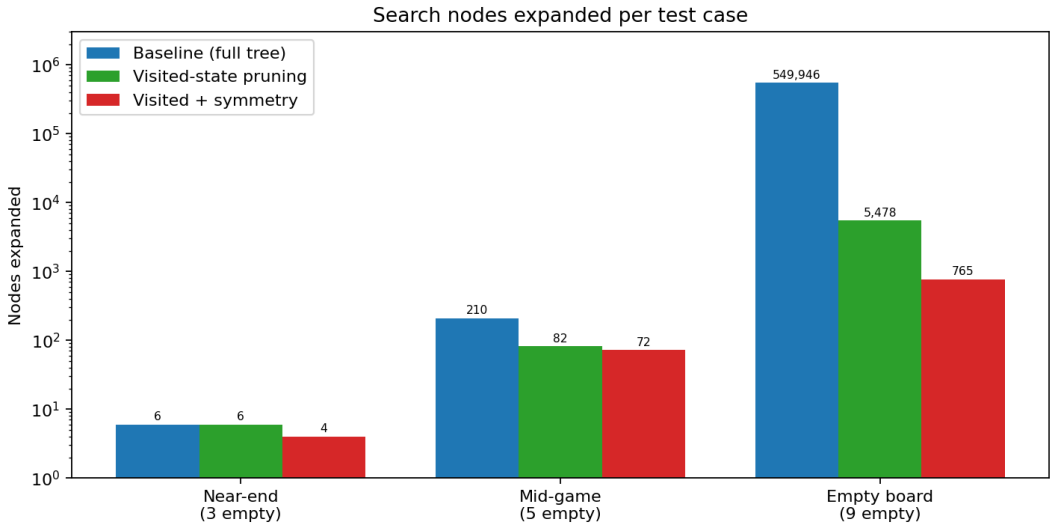


Figure 6: Nodes expanded per test case for the baseline and the two pruning stages (log scale)

8. AI Contribution Note

- **AI tools used (if any):** ChatGPT (SLE-2 implementation and report); Claude (SLE-3 performance redesign, diagrams and report update).
- **What AI helped with:** understanding the SLE requirements, preparing/refining the BFS and DFS implementation, explaining profiling concepts and organizing the report; in SLE-3, designing and

implementing the bitboard, visited-state and symmetry optimizations, running the baseline-vs-optimized benchmark, drawing the C4 diagrams and updating the results.

- **What I did myself:** ran the SLE-2 Python program, collected its terminal results (Section 7.1), and used them to prepare the comparison. The SLE-3 figures in Section 7.2 were measured by the AI tool in its own environment.

9. Conclusion

The C4 model shows how the same Tic-Tac-Toe BFS/DFS system can be described at different levels of detail without losing consistency. At the Context level the system is simply a board-to-move search agent, and the search algorithm is hidden completely. At the Container and Component levels the input, search engine, visited set, frontier, terminal test and move selector become visible, and BFS and DFS look identical apart from one detail — a queue versus a stack. At the Code level the exact search and measurement functions are named.

SLE-3 performance result: visited-state pruning and symmetry reduction cut the empty-board search from 549,946 to 765 expanded nodes (about 719× fewer) and the runtime from about 881 ms to 3.2 ms for BFS (273× faster) and from about 890 ms to 3.4 ms for DFS (259× faster), with the same move and value selected. The mid-game position improved by about 2.5×. For the 3-empty position there was no gain (about 0.02 ms either way, within timing noise) because the tree is already tiny. In this re-run the baseline BFS and DFS times were nearly equal (881 ms and 890 ms), and after optimization both expand the same set of states, so the DFS advantage seen in SLE-2 should be treated as specific to that machine and run. All results apply to the tested positions and implementation.